Modelo 1:https://huggingface.co/pysentimiento/robertuito-sentiment-analysis
Modelo 2:https://huggingface.co/nlptown/bert-base-multilingual-uncased-sentiment

```
frases_prueba = [
    "¡Excelente atención, la comida llegó rapidísimo y estaba riquísima!",  # 1. Positivo claro
    "El lugar es horrendo, la comida fría y encima carísimo. No vuelvo más.",  # 2. Negativo claro
    "Bueno, al menos el vaso estaba limpio, supongo...",                    # 3. Ironía / Neutro-Negativo
    "Zarpado en rico el tostado, re recomiendo este lugar.",                # 4. Jerga argentina / Positivo
    "Meh, zafa.",                                                           # 5. Texto muy corto / Neutro
    "The service was terrible and the waiting time was endless.",           # 6. Otro idioma (Inglés) / Negativo
    "El mozo tardó dos horas pero el postre casi compensa la tragedia.",    # 7. Mixto / Irónico
    "Estuvo bien."                                                          # 8. Neutro / Corto simple
]
```

```
Cargando pipeline para: pysentimiento/robertuito-sentiment-analysis
Loading weights: 100% 201/201 [00:00<00:00, 3723.48it/s]
```

	frase	etiqueta	puntaje	modelo
0	¡Excelente atención, la comida llegó rapidísim...	POS	0.9784	RoBERTuito
1	El lugar es horrendo, la comida fría y encima ...	NEG	0.9800	RoBERTuito
2	Bueno, al menos el vaso estaba limpio, supongo...	NEU	0.5389	RoBERTuito
3	Zarpado en rico el tostado, re recomiendo este...	POS	0.9574	RoBERTuito
4	Meh, zafa.	NEG	0.5360	RoBERTuito
5	The service was terrible and the waiting time ...	NEG	0.9308	RoBERTuito
6	El mozo tardó dos horas pero el postre casi co...	NEU	0.7015	RoBERTuito
7	Estuvo bien.	POS	0.8318	RoBERTuito

	frase	etiqueta	puntaje	modelo
0	¡Excelente atención, la comida llegó rapidísim...	5 stars	0.8376	Multilingual BERT
1	El lugar es horrendo, la comida fría y encima ...	1 star	0.6203	Multilingual BERT
2	Bueno, al menos el vaso estaba limpio, supongo...	3 stars	0.5299	Multilingual BERT
3	Zarpado en rico el tostado, re recomiendo este...	5 stars	0.6485	Multilingual BERT
4	Meh, zafa.	2 stars	0.4587	Multilingual BERT
5	The service was terrible and the waiting time ...	1 star	0.6433	Multilingual BERT
6	El mozo tardó dos horas pero el postre casi co...	3 stars	0.4298	Multilingual BERT
7	Estuvo bien.	4 stars	0.4050	Multilingual BERT

```
--- ANÁLISIS DE PIPELINE ---
Tipo de dato devuelto (type): <class 'list'>
Estructura completa: [{'label': 'POS', 'score': 0.9786208868026733}]
Acceso a la etiqueta principal: POS
Acceso al puntaje de confianza: 0.9786208868026733

--- ANÁLISIS DE INFERENCE CLIENT ---
Tipo de dato devuelto (type): <class 'list'>
Estructura completa: [{'label': 'POS', 'score': 0.9786208868026733}]
Acceso a la etiqueta principal: POS
Acceso al puntaje de confianza: 0.9786208868026733
```

USO DE IA GENERATIVA:

A la hora de usar código se usó parte para generar el entorno virtual asi como la interfaz de grado. En colab se modificó para se puedan usar los dos modelos

A. El problema

1. ¿Qué problema querés resolver y para quién? ¿Por qué un modelo de IA y no unas reglas escritas a mano (un if con palabras clave)?

fue hecho pensando en restaurantes para llegar mejor a sus cliente.
Se usa el modelo de IA para procesar la complejidad del idioma, sino la reseñas se tendría que leer si o no, ya que esta puede llegar a comprender la ironia o jergas

B. El modelo

2. ¿Qué modelos consideraste y por qué te quedaste con uno? Citá lo que leíste en su model card: tarea, etiquetas, idiomas, datos de entrenamiento, tamaño de los pesos.

Como el objetivo era leer reseñas se usó unos modelos de análisis de sentimientos llamado BERT ya se usa Transformers el primero(robertuito) se se destino para comentarios de twitter y el segundo(bert-base-multilingual-uncased-sentiment) para reviews de productos.

Robertuito:
-Tarea: Clasificacion de texto y análisis de sentimiento

-Etiqueta: positivo, negativo y neutro

-Datos entrenamiento: Conjuntos de datos gigante alrededor de 5.000 tuits(llamada TASS 2020) para el análisis de sentimiento

-Tamaño pesos: el modelo está basado en roberta por que tendria una cantidad de parametros considerables(110 millones aprox)

el otro se descarto aunque sea similar los datos que se es español es menor y se pierde las ironia o jerga

3. ¿Dónde se equivoca? Mostrá ejemplos reales de tu notebook y explicá por qué creés que se equivoca, relacionándolo con los datos con los que fue entrenado.

Una de las frases usada "Bueno, al menos el vaso estaba limpio, supongo..."
los 2 modelos lo ven como neutra positiva cuando no lo es

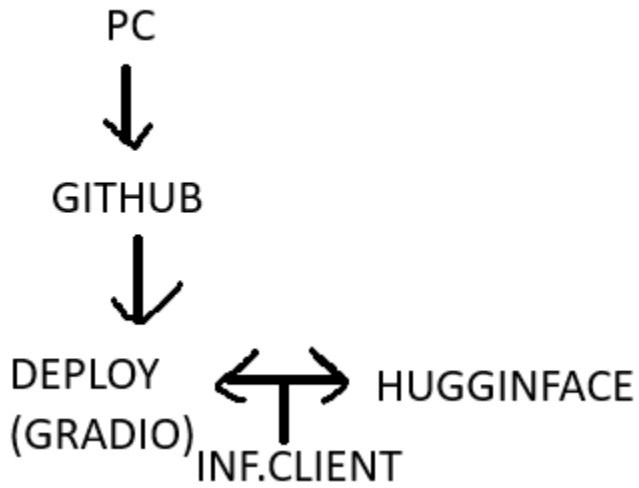
Bueno, al menos el vaso estaba limpio, supongo...	NEU	0.5389	RoBERTuito

al ver la palabra limpio lo ve como algo bueno la valoración “al menos es” una reacción positiva ,esta es una de las falla ocurrentes de estos modelos

4. Tu app, ¿está entrenando o infiriendo? Explica la diferencia con tu propio caso.

está infiriendo ya que se usó un modelo ya entrenados, si queremos modificar este entrenamiento deberíamos cambiar los pesos de la red neuronal

C. Dónde corre cada cosa
5. ¿Dónde vive el código y dónde vive el modelo? Hacé un esquema simple (cajas y flechas, a mano o en cualquier herramienta) con los lugares que intervienen en tu app: tu compu, GitHub, la plataforma de deploy y Hugging Face. En cada caja anotá qué hay ahí, y marcá con flechas qué viaja de un lugar a otro cuando alguien usa la app. Tiene que quedar claro: ¿dónde está guardado tu código? ¿Dónde se ejecuta? ¿Dónde están los pesos del modelo y dónde se hace la inferencia?



Cuando el usuario ejecuta el programa y escribe una reseña y mediante el InferenceClient llama a los servidores de hugging face y este valora la reseña y devuelve el mismo

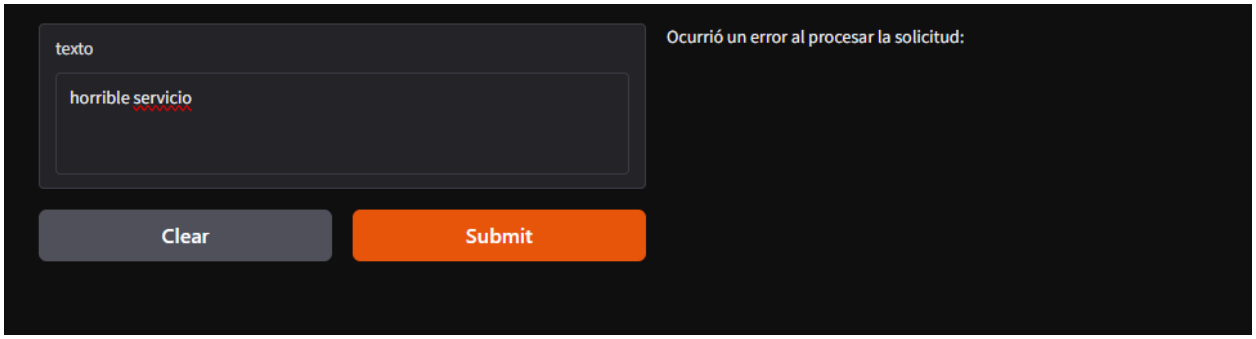
6. ¿Usaste pipeline o InferenceClient en la app publicada? Justificalo con números (memoria del plan gratuito, peso del modelo, tiempo de respuesta). ¿Qué pasaría si hubieras elegido la otra?

se probó con los dos modelos pero el mejor seria el InferenceClient ya que este pasa todo el procesamiento al servidor de huggin face cosa que en pipeline no pasa ya que está la carga se la llevaría el servidor deploy consumiendo toda la memoria

```
Tiempo con pipeline local: 1.1271 segundos
```

D. El deploy
7. ¿Dónde deployaste y por qué ahí? Compará tu elección con al menos otras dos opciones vistas en clase (hosting compartido, VPS, PaaS, serverless, nubes elásticas) y explicá por qué las descartaste. ¿Por qué tu app necesita un servidor que quede encendido?

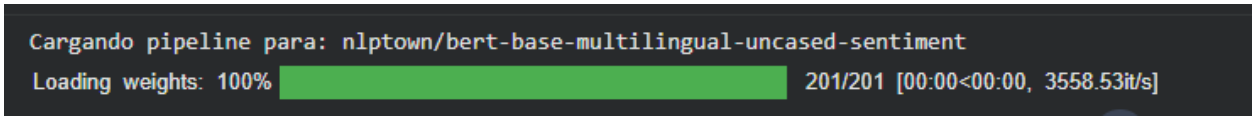
metodo local del InferenceClient



El deploy se realizó con Huggin face para en la nube para que el procesos se realizará en los servidores. Al acabarse los tokens se probó el método local que funcionó perfectamente y se puede realizar la veces que quiera

E. Lo que se rompió
8. Documentá al menos dos errores reales que te aparecieron durante el trabajo. Para cada uno:
• el mensaje de error completo (captura o texto copiado);
• dónde apareció: en tu compu, en la plataforma de deploy o en Hugging Face;
• qué lo causaba;
• cómo lo resolviste, paso a paso, como para que otra persona pueda seguirlo.

A la hora de crear el programa no se han visto muchos errores. La mayoría por falta de pasos que se resolvieron con los materiales ofrecidos en la materias



En cierto punto uno de los modelos dejó de funcionar, los pesos cargan pero se quedan congelados procesando infinitamente consumiendo muchos recursos locales. Por falta de tiempo no se le vio solucion

9. Límites de tu solución: ¿qué pasa si la usan 1000 personas a la vez? ¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)? ¿Qué cambiarías para que sea un producto real?

Si la usarán 1000 personas necesitaría un servidor propio armado o un plan para procesar todas las solicitudes entrantes, además de la interfaz y la dirección que lleva esta como, dar un lugar dependiendo de tus preferencias ej: “quiero un lugar de películas baratos y con buena atención” y te mostrarán lugares con esa especificaciones.
F. Vocabulario
10. Tomá 5 líneas de tu app.py y señalá en ellas: una clase, una instancia, un método, un atributo, un parámetro y su argumento, y un valor de retorno.

```
def clasificar_resenia(texto):

    if not texto.strip():
        return "Por favor, ingresa un texto válido para analizar."

    try:
                resultado = client.text_classification(texto)
 respuesta_formateada = ""
        for item in resultado:
            etiqueta = item.get('label')
            puntaje = item.get('score')
            respuesta_formateada += f"**Etiqueta:** {etiqueta} |
**Confianza:** {puntaje:.4f}\n"

        return respuesta_formateada

    except Exception as e:
        return f"Ocurrió un error al procesar la solicitud: {str(e)}"
```

en sí mismo la aplicación no contiene una clase. Tiene una función que recibe por parámetro el contenido de la textbox de gradio el condicional se encarga que el texto sea válido que no esté en blanco, luego se usa el try para conectar con el servidor de Huggin face el método resultado ya clasifica el texto recibido y se le da el puntaje si esto no pasa tira una excepcion